



הצעת פרויקט – אדם זבולון

פרטי מגיש ההצעה

- שם סטודנט: אדם זבולון
- תעודת זהות: 329441273
- שם מכללה: מקיף ה דרכא אשקלון
- שם מנחה: גל בראון
- סמל מוסד: 644450
- תאריך הגשה: 04.01.2026
- שם פרויקט:
"מערכת שיתוף קבצים מבוססת בסגנון BitTorrent"

תוכן עניינים

1.....	הצעת פרויקט – אדם זבולון
1.....	פרטי מגיש ההצעה
3.....	תיאור הנושא
3.....	תיאור הפרויקט
3.....	מנוע BitTorrent בפיתוח
4.....	ממשק משתמש גרפי ב-Java
4.....	אינטגרציה בין החלקים
4.....	2. הגדרת הבעיה האלגוריתמית
5.....	3. רקע תיאורטי
5.....	3.1 P2P לעומת Client-Server
5.....	3.2 מבנה מערכת BitTorrent
6.....	3.3 אלגוריתמים מרכזיים ב-BitTorrent
6.....	3.4 Use Cases במערכת BitTorrent המוצעת
7.....	4. ניתוח מצב קיים ופתרונות קיימים
8.....	5. הפתרון המוצע – ארכיטקטורה ושתי השפות
8.....	5.1 חלוקה לשכבות ולמודולים
8.....	תרשים ארכיטקטורה
8.....	5.2 מודולים עיקריים בפיתוח
10.....	5.3 מודולים עיקריים ב-Java
10.....	6. תיאור פרוטוקולי תקשורת
10.....	6.1 תקשורת עם ה-Tracker – HTTP Tracker Protocol
11.....	6.2 תקשורת בין peers – Peer Wire Protocol
12.....	תרשים רצף הורדה
12.....	7. אלגוריתמיקה – פירוט ו-SWOT
12.....	7.1 rarest-first (בחירת חלקים)
14.....	7.2 Tit-for-Tat (choke/unchoke) – בחירת peers
15.....	7.3 Random Piece / Round-Robin – השוואה למדיניות בסיסית
15.....	8. אבטחת מידע
15.....	8.1 Hash per Piece – שלמות מידע
15.....	8.2 התמודדות עם peers זדוניים או שגויים
16.....	8.3 פרטיות מול ה-Tracker
16.....	9. פיתוחים עתידיים
17.....	10. מסד נתונים (JSON + SQLite) – Persistency
17.....	שמירת מצב הורדה בזמן אמת – JSON
17.....	היסטוריה וסטטיסטיקות – SQLite
18.....	11. לוח זמנים
19.....	12. רשימת מאמרים (אלגוריתמיקה ו-BitTorrent)
20.....	13. מילון מושגים

תיאור הנושא

נושא הפרויקט עוסק בפרוטוקול BitTorrent וברשתות שיתוף קבצים מבזרות (Peer-to-Peer). בניגוד למודל Client–Server קלאסי, שבו קובץ יורד משרת מרכזי אחד, ברשת BitTorrent כל משתמש (peer) משתתף גם בהורדה וגם בהפצה של חלקי הקובץ. הקובץ מחולק לחלקים קטנים (pieces), וכל peer יכול להוריד חלקים שונים ממספר peers במקביל, ובזמנית לשתף חלקים שכבר נמצאים ברשותו.

העבודה ברשת מתבססת על קובצי torrent, שהם קובצי מטא-דאטה המקודדים בפורמט Bencode. קבצים אלו אינם מכילים את המידע עצמו, אלא מתארים את מבנה הקובץ, גודל החלקים, רשימת hash-ים (SHA-1) לכל חלק, וכתובת של tracker. ה-tracker משמש כשרת תיווך שמחזיר ללקוח רשימת peers פעילים המשתתפים באותו torrent.

כדי לייעל את ההפצה ולשמור על זמינות המידע, BitTorrent עושה שימוש באלגוריתמים ייעודיים. אלגוריתם rarest-first נועד למנוע מצב שבו חלקים נדירים "נעלמים" מהרשת, בעוד שמנגנון choke/unchoke, המבוסס על עקרון Tit-for-Tat, יוצר תמריץ לשיתוף הוגן בין המשתתפים. שילוב מנגנונים אלו מאפשר ניצול יעיל של רוחב הפס, קיצור זמני ההורדה ושמירה על יציבות הסוור.

תיאור הפרויקט

פרויקט זה יהיה מורכב משני חלקים עיקריים, הפועלים יחד כמערכת אחת שלמה: מנוע BitTorrent בצד השרת וממשק משתמש גרפי בצד הלקוח.

מנוע BitTorrent בפייתון

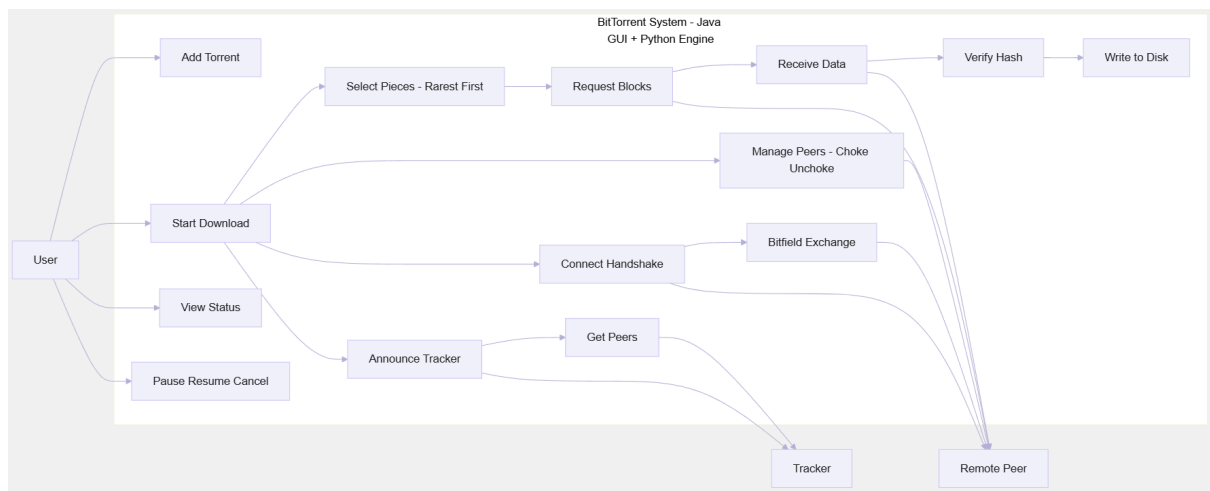
החלק הראשון הוא מנוע BitTorrent שכתוב בפייתון. זהו החלק שבו מתבצעת כל העבודה "האמיתית" של הפרוטוקול. המנוע קורא ומנתח קבצי torrent, שהם קבצי מטא-דאטה המקודדים בפורמט Bencode, וכוללים מידע כגון שם הקובץ, גודל כל חלק (piece length), רשימת ה-hash-ים (SHA-1) לכל חלק וכתובת ה-tracker.

לאחר קריאת קובץ ה-torrent וחישוב ה-info_hash, המנוע שולח בקשת HTTP/HTTPS ל-tracker המוגדר בקובץ. בבקשת ה-announce מועברים פרמטרים כמו port, peer_id, info_hash, כמות הנתונים שהורדו והועלו (downloaded, uploaded, left), ואירועי מצב כמו started או completed. בתגובה, ה-tracker מחזיר רשימת peers פוטנציאליים יחד עם מידע על מספר ה-seeders וה-leechers בסוור.

בהמשך, המנוע מתחבר אל ה-peers באמצעות TCP ומבצע מולם את ה-handshake המוגדר ב-peer wire protocol של BitTorrent. ההתחברות כוללת שליחת המחרוזת "BitTorrent protocol", שדות דגלים, ה-info_hash וה-peer_id. לאחר ההנדשייק, התקשורת ממשיכה כזרם של הודעות length-prefixed כגון choke, unchoke, interested, have, bitfield, request, piece ועוד.

בתוך המנוע ימומשו שני אלגוריתמים מרכזיים. אלגוריתם rarest-first מחליט אילו חלקים (pieces) של הקובץ כדאי להוריד קודם, ומעדיף את החלקים הנדירים ביותר ברשת. במקביל, אלגוריתם choke/unchoke (Tit-for-Tat)) קובע מול אילו peers כדאי "להיפתח" ולתת להם נתונים, על בסיס התרומה שלהם להורדה.

בנוסף, כל piece שמתקבל עובר בדיקת תקינות (hash per piece) באמצעות השוואת ה-SHA-1 שלו לערך שמופיע בקובץ ה-.torrent. רק אם הבדיקה מצליחה, הנתונים נכתבים לקובץ הסופי.



ממשק משתמש גרפי ב-Java

החלק השני של המערכת הוא ממשק משתמש גרפי בשפת Java, באמצעות JavaFX או Swing. מטרת הממשק היא לאפשר עבודה נוחה עם מנוע ההורדות ולא להסתפק בשורת פקודה בלבד. דרך ה-GUI ניתן לבחור קובצי torrent, להוסיף אותם לרשימת ההורדות, ולצפות עבור כל הורדה בפרטים כגון שם הקובץ, גודל, אחוז התקדמות, מהירות הורדה ומספר peers מחוברים.

הממשק מאפשר גם שליטה ישירה בהורדות, כולל פעולות Pause, Resume ו-Cancel עבור כל torrent בנפרד.

אינטגרציה בין החלקים

כדי לחבר בין מנוע ה-Python ל-GUI ב-Java, יפותח שרת HTTP מקומי (REST API) שרץ על localhost. השרת יחשוף פעולות להתחלת הורדה חדשה, קבלת סטטוס כל ההורדות בפורמט JSON, ושליטה בהורדה מפציפת באמצעות pause, resume ו-cancel.

ה-GUI ב-Java משתמש ב-HttpClient כדי לתקשר עם ה-API, להציג את הנתונים למשתמש ולהפעיל את הפקודות. בצורה זו הפרויקט משלב שתי שפות תכנות שונות: Python כמנוע רשת ואלגוריתמיקה, ו-Java כשכבת תצוגה וחווית משתמש, עם אינטגרציה ביניהן דרך API.

2. הגדרת הבעיה האלגוריתמית

ברמת האלגוריתמיקה, נתמקד בשתי שאלות מרכזיות:

1. איזה חלק (piece) של הקובץ כדאי להוריד עכשיו?

הקובץ מחולק לעשרות או מאות חלקי קטנים. חלק מהם מופיעים אצל המון peers (חלק "פופולרי"), וחלק מהם נדירים – יש רק אצל מעט peers. אם נבחר חלקים בצורה נאיבית (למשל אקראית), יכול להיווצר מצב שבו לקראת סוף ההורדה נשארים "חלקים נדירים" שקשה להשיג, ואם ה-peer האחרון שמחזיק אותם ייעלם – אין איך להשלים את הקובץ. כאן נכנס אלגוריתם rarest-first, שמנסה מראש לטפל בבעיה הזו.

2. מאילו peers כדאי לי לקבל נתונים?

אנחנו לא רוצים לחלק את רוחב הפס באופן עיוור בין כל מי שמחובר אליי. חלק מה-peers יכולים להיות איטיים או בכלל לא להעלות בחזרה. כאן נכנס אלגוריתם choke/unchoke, שנועד לתגמל peers שתורמים לי נתונים, ולהפחית או "לחנוק" peers שלא משתפים פעולה.

אני מנסח את הבעיה האלגוריתמית שלי כך:

אני רוצה לתכנן וליישם אלגוריתמים לבחירת חלקים (piece selection) ולבחירת עמיתים (peer selection) ברשת BitTorrent, כך שזמן ההורדה יהיה קצר ככל האפשר, רוחב הפס ינוצל ביעילות, וכל חלקי הקובץ יישארו זמינים בסוורם (robustness). כל זה צריך לקרות תוך שמירה על התמריצים לשיתוף (מי שתורם מקבל יותר) ותוך הבטחת תקינות המידע (hash per piece) והגנה בסיסית מפני peers בעייתיים.

בפרויקט איישם בפועל:

- **rarest-first** כאלגוריתם הבחירה של pieces,
- **(choke/unchoke) (Tit-for-Tat)** כאלגוריתם הבחירה של peers,

ואשווה אותם למדיניות בסיסית יותר (כמו בחירה אקראית של pieces ו-round-robin על peers) כדי להדגים את תרומתם לביצועים.

3. רקע תיאורטי

3.1 P2P לעומת Client–Server

במודל Client–Server, קובץ יורד משרת מרכזי אחד (למשל שרת HTTP). אם אלף אנשים מורידים את אותו קובץ, כל העומס נופל על השרת. הוא חייב רוחב פס גדול, ואם הוא נופל – אין הורדה.

במודל Peer-to-Peer (P2P), כמו ב-BitTorrent, אין שרת אחד שמחזיק את כל הקובץ. במקום זה, כל עמית (peer) יכול להחזיק חלק מהקובץ ולהעביר אותו הלאה ל-peers אחרים. כך כל מי שמוריד הופך גם לסוג של "שרת קטן", והעומס מתפזר על פני כל רשת העמיתים.

3.2 מבנה מערכת BitTorrent

ב-BitTorrent, כל הורדה מבוססת על כמה רכיבים:

- **קובץ torrent** – זהו קובץ מטא-דאטה בפורמט Bencode, שמכיל:
 - מידע על הקובץ או הקבצים (שמות, גדלים),
 - גודל כל חלק (piece length),
 - רשימת (hashes (SHA-1 לכל piece,
 - וכתובת או כמה כתובות של trackers.
- **Tracker** – שרת ייעודי שמנהל עבור כל `info_hash` רשימה של peers שמשתתפים בהורדה. כאשר הלקוח שלי מבצע announce, הוא מקבל מה-tracker רשימת (IP ו-port) שאיתם כדאי לו לנסות להתחבר. ה-tracker לא מעביר את התוכן עצמו, רק "משדך" בין peers.
- **Peer Wire Protocol** – זה הפרוטוקול שרץ מעל TCP בין שני לקוחות BitTorrent. הוא מתחיל ב-handshake, ואחריו מתנהלת סדרה של הודעות כמו `interested`, `unchoke`, `choke`, `not interested`, `have`, `bitfield`, `request`, `piece` וכן הלאה.

3.3 אלגוריתמים מרכזיים ב-BitTorrent

בספרות המחקר על BitTorrent מדברים הרבה על שני אלגוריתמים:

- **Rarest-First** – אלגוריתם לבחירת pieces. הוא שומר סטטיסטיקה לגבי כמה peers מחזיקים כל piece, ובוחר קודם את החלקים הנדירים ביותר. מחקרים הראו שזה מפחית את הסיכון שחלקים מסוימים ייעלמו ומוביל לביצועים טובים יותר מאשר בחירה אקראית.
- **(Choke / Unchoke (Tit-for-Tat** – אלגוריתם לבחירת peers. כל לקוח מעדיף לפתוח את רוחב הפס שלו ל-peers שתורמים לו יותר (מעלים אליו data). העיקרון דומה ל-Tit-for-Tat מתורת המשחקים: "אם אתה נחמד אליי, אני נחמד אליך".

3.4 Use Cases במערכת BitTorrent המוצעת

בהתבסס על מבנה מערכת BitTorrent והאלגוריתמים שתוארו קודם, ניתן להגדיר מספר מקרי שימוש מרכזיים (Use Cases) המתארים את האינטראקציה בין המשתמש, המערכת והרכיבים החיצוניים.

השחקן הראשי במערכת הוא **המשתמש**, אשר מפעיל את המערכת דרך ממשק המשתמש הגרפי. בנוסף קיימים שחקנים משניים שאינם משתמשים אנושיים, כגון **Tracker** ו-**Peers**, אשר מתקשרים עם המערכת כחלק מתהליך ההורדה.

מקרי השימוש המרכזיים במערכת הם:

- **הוספת קובץ torrent** – המשתמש בוחר קובץ torrent. דרך ה-GUI ומבקש להתחיל הורדה חדשה.
- **התחלת הורדה** – המערכת מנתחת את קובץ ה-torrent, מחשבת info_hash, ומבצעת announce ל-tracker.
- **קבלת רשימת peers** – המערכת מקבלת מה-tracker רשימת peers זמינים ומידע על הסוורם.
- **חיבור ל-peers** – המערכת יוצרת חיבורי TCP ל-peers ומבצעת handshake לפי Peer Wire Protocol.
- **הורדת חלקי קובץ (pieces)** – המערכת מבקשת ומקבלת חלקים מה-peers, בהתאם לאלגוריתם rarest-first.
- **ניהול (peers choke/unchoke)** – המערכת בוחרת אילו peers יקבלו רוחב פס בהתאם לאלגוריתם Tit-for-Tat.
- **בדיקת תקינות נתונים** – כל piece שהתקבל עובר בדיקת hash לפני כתיבה לקובץ.
- **הצגת סטטוס הורדה** – המשתמש צופה בנתוני ההורדה דרך ה-GUI (אחוזים, מהירות, peers).
- **שליטה בהורדה** – המשתמש יכול לבצע פעולות Pause, Resume ו-Cancel על הורדה פעילה.

4. ניתוח מצב קיים ופתרונות קיימים

לקוחות BitTorrent פופולריים כמו qBittorrent ו-Transmission משתמשים בגרסאות של rarest-first ו-choke בתור ברירת מחדל, ומוסיפים עליהם עוד לוגיקה ו-tuning.

במקביל, יש מחקרים שהציעו אלגוריתמים מורכבים יותר (כמו network coding או מודלים כלכליים מתקדמים), אבל מאמרים כמו *Rarest First and Choke Algorithms Are Enough* מראים שבמקרים רבים אין רווח משמעותי מספיק כדי להצדיק את המורכבות.

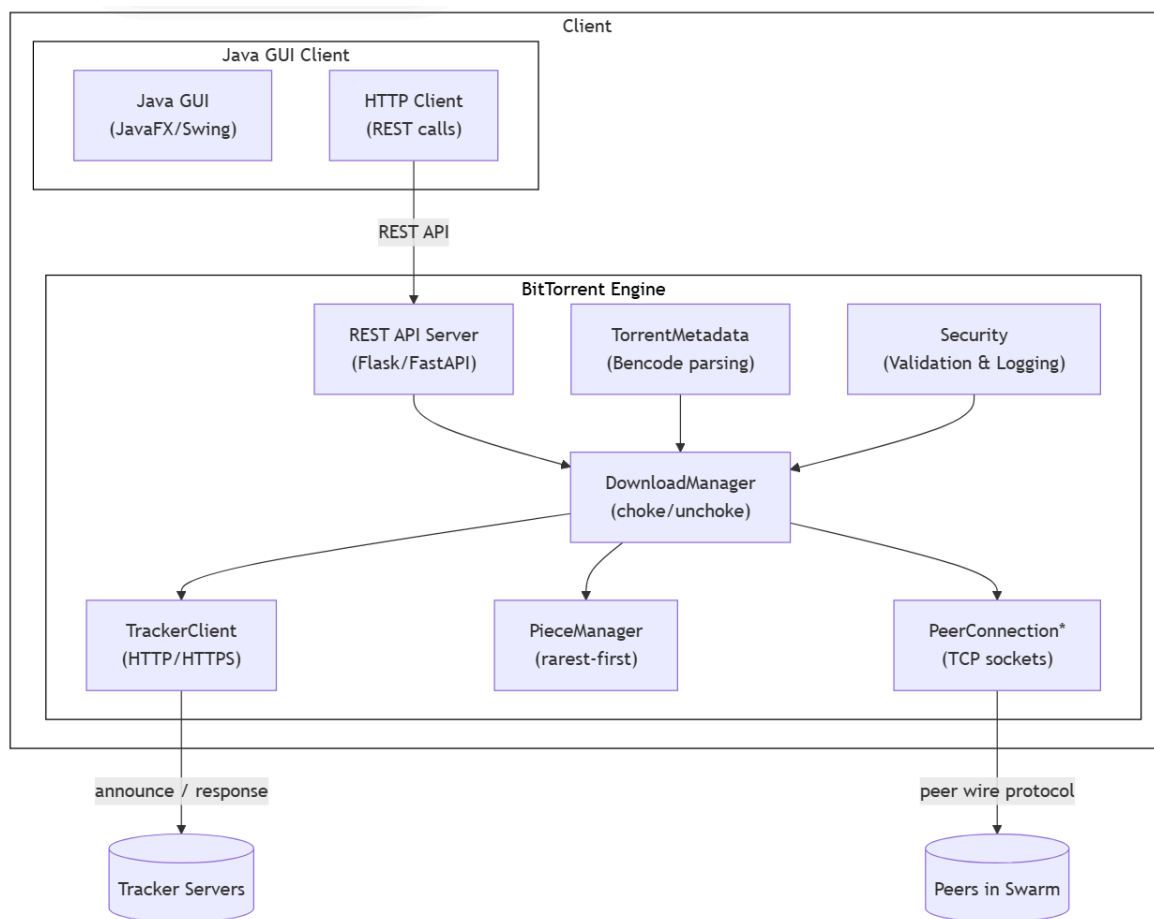
בפרויקט שלי אני מתכוון ליישם את האלגוריתמים הקלאסיים, וגם להשוות אותם למדיניות פשוטה כמו Random Piece Selection ו-Round-Robin בין peers. ההשוואה תעשה בעזרת ניסויים בפועל (מדידת זמן הורדה, מהירות ממוצעת וכו'), ובאמצעות טבלאות SWOT לכל אלגוריתם.

5. הפתרון המוצע – ארכיטקטורה ושתי השפות

5.1 חלוקה לשכבות ולמודולים

אני מתכנן לבנות את הפרויקט בצורה מודולרית ומופרדת לשכבות, גם בצד הפיתוח וגם בצד ה-Java.

תרשים ארכיטקטורה



5.2 מודולים עיקריים בפייתון

- `bencode.py` – מימוש `encode/decode` ל-Bencode (מספרים, מחרוזות, רשימות ומילונים).
- `torrent_metadata.py` – מחלקה `TorrentMetadata` שקוראת קובץ `.torrent`, מפענחת את ה-Bencode, ושומרת את הנתונים הרלוונטיים (`announce`, `piece length`, רשימת `hashes` וכו'). כאן גם מחושב `info_hash` כ-SHA-1 על המילון `info` המקודד שוב ב-Bencode.
- `tracker_client.py` – אחראי על שליחת בקשות `announce` ל-tracker וקבלת התשובה:
 - הוא שולח GET עם פרמטרים כמו `info_hash`, `peer_id`, `port`, `uploaded`, `downloaded`, `left` ו-`event`.
 - התגובה שמתקבלת ב-Bencode מפוענחת לרשימת `peers` ומידע נוסף.
- `peer_connection.py` – מנהל חיבור TCP ל-peer אחד:
 - מבצע `handshake` לפי BEP-3,

- מקבל ושולח הודעות כמו `bitfield`, `have`, `interested`, `request`, `piece`, `choke`, `unchoke`.
- `piece_manager.py` – שומר טבלת מצב לכל `piece`:
 - האם הורדנו אותו,
 - כמה `peers` מחזיקים אותו,
 - ומממש את אלגוריתם `rarest-first` לבחירת ה-`piece` הבא להורדה.
- `download_manager.py` – הלוגיקה המרכזית שמנהלת את ההורדה:
 - פתיחת חיבורים למספר `peers`,
 - ניהול `choke/unchoke` מול כל `peer`,
 - ניהול תור הבקשות (`requests`) לחלקים ולבלוקים.
- `security.py` – מטפל בבקורות אבטחה:
 - בדיקות SHA-1 לכל `piece`,
 - ניהול `timeouts`,
 - זיהוי וניתוק `peers` שמפרים את ה-`state machine` או שולחים הודעות לא חוקיות.
- `api_server.py` – שרת HTTP מקומי (למשל `Flask` או `FastAPI`) שמספק את ה-REST API שה-GUI ב-Java משתמש בו.

5.3 מודולים עיקריים ב-Java

- מחלקה `TorrentClientGUI` – החלון הראשי:
 - יציג טבלה של הורדות (שם קובץ, גודל, אחוז התקדמות, מהירות, `peers` מחוברים),

○ ויכלול כפתורים כמו "הוסף torrent", "התחל", "Pause/Resume" ו-"בטל".

● מחלקה `ApiService` – לקוח HTTP שמדבר עם ה-REST API בפיתון:

○ `startDownload(File torrentFile) -> torrentId`

○ `<getStatus() -> List<TorrentStatus`

○ `(pause(torrentId), resume(torrentId), cancel(torrentId`

ה-GUI ב-Java אחראי לתצוגה ולחויית המשתמש, והמנוע בפיתון אחראי על הלוגיקה של BitTorrent והתקשורת ברשת.

6. תיאור פרוטוקולי תקשורת

6.1 תקשורת עם ה-HTTP Tracker Protocol – Tracker

כשאני מתחיל הורדה, אני שולח בקשת GET לכתובת ה-announce של ה-tracker (שנלקחת מתוך קובץ ה-`torrent`). בבקשה הזו מופיעים, בין השאר:

● `info_hash` – SHA-1 של המילון `info`.

● `peer_id` – מזהה ייחודי בגודל 20 בתים.

● `port` – הפורט שבו אני מאזין.

● `uploaded, downloaded, left` – כמה העליתי, כמה הורדתי, וכמה נשאר.

● `event` – למשל `started, completed`, או `stopped`.

ה-tracker מחזיר תגובה שמקודדת ב-Bencode ובה בין השאר:

● `interval` – כמה זמן לחכות עד ה-announce הבא.

● `complete` ו-`incomplete` – כמה seeders ו-leechers יש.

● `peers` – רשימת `peers`, או בפורמט רגיל (IP+port לכל אחד) או בפורמט `compact` (מבנה בינארי)

6.2 תקשורת בין Peers – Peer Wire Protocol

אחרי שיש לי כתובות של `peers`, אני מתחבר אליהם ב-TCP ועובד לפי הפרוטוקול הבא:

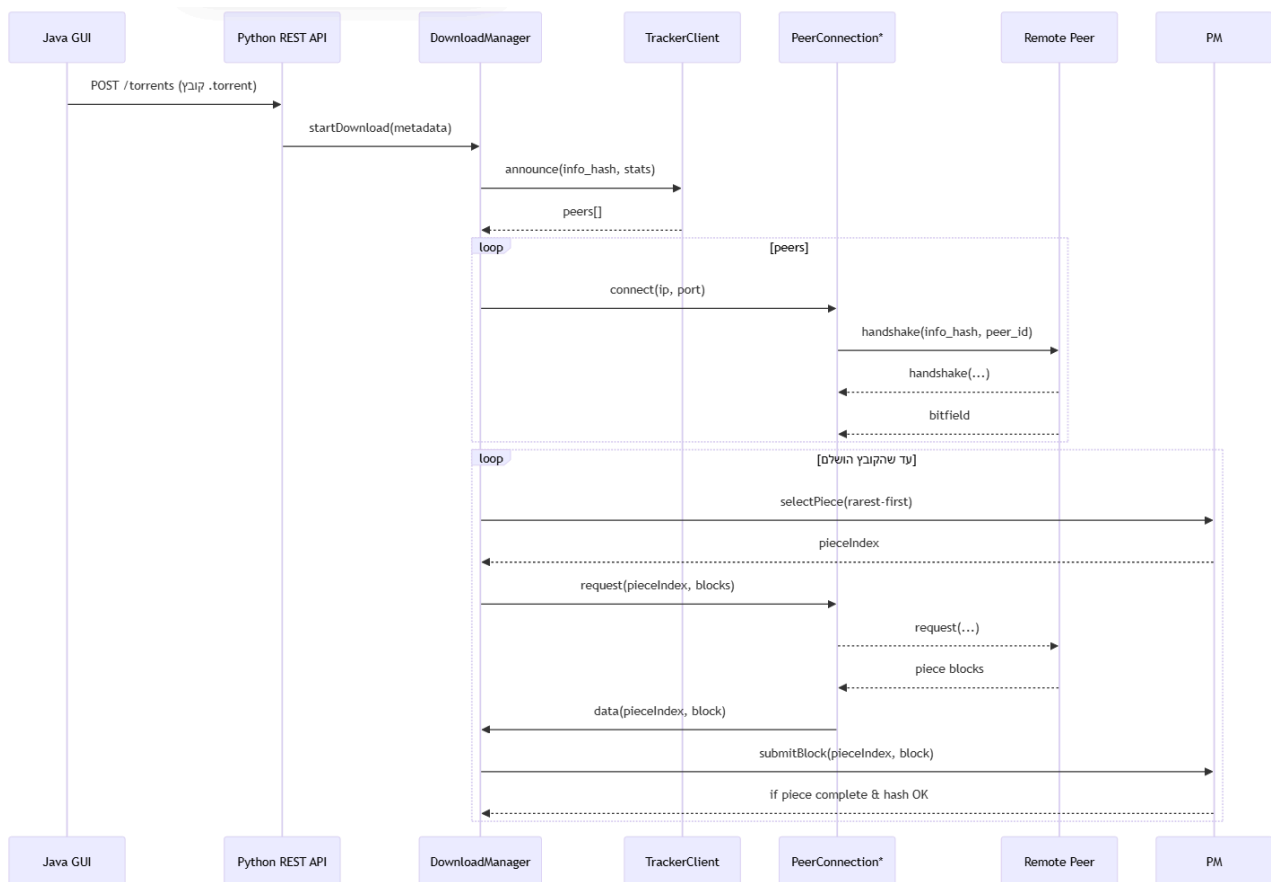
● Handshake – שליחת:

- אורך מחרוזת הפרוטוקול (19),
- המחרוזת "BitTorrent protocol",
- 8 בתים של דגלים (reserved),
- `info_hash`,
- `peer_id`.

● לאחר ה-Handshake – אני מקבל ושולח הודעות length-prefixed:

- הודעות מצב – `choke`, `unchoke`, `interested`, `not interested`.
- הודעות מידע – `have`, `bitfield`.
- הודעות נתונים – `request`, `piece`, `cancel`.

תרשים רצף הורדה



התרשים מתאר בצורה מרוכזת את כל תהליך ההורדה בטורנט מרגע שהמשתמש מתחיל אותה ב-Java GUI ועד שהחלקים של הקובץ מתקבלים בפועל מה-`peer`. כשמשתמש בוחר קובץ `torrent`, ה-GUI שולח את הבקשה ל-Python REST API, שמפעיל את ה-`DownloadManager`.

המנהל הזה פונה ל-TrackerClient כדי לבצע announce, ומקבל ממנו רשימת peers זמינים. עבור כל peer כזה נוצר חיבור TCP, מתבצע Handshake לפי פרוטוקול BitTorrent, וה-peer מחזיר Bitfield שמראה אילו חלקים של הקובץ יש לו. לאחר מכן מתחילה לולאה שבה ה-DownloadManager מבקש מה-PieceManager לבחור piece להורדה (לפי rarest-first), שולח בקשות ל-peer לקבלת הבלוקים של אותו piece, מקבל את הנתונים בחזרה ושולח אותם ל-PieceManager להרכבה ובדיקת hash. התהליך הזה חוזר על עצמו עד שכל החלקים נאספים בהצלחה והקובץ מושלם.

7. אלגוריתמיקה – פירוט ו-SWOT

7.1 אלגוריתם rarest-first (בחירת חלקים)

בזמן שההורדה רצה, אני אוסף מידע על אילו pieces קיימים אצל כל peer. זה נעשה דרך הודעות bitfield (שמגיעות בתחילת החיבור) ו-have (שמודיעות על חידוש). עבור כל piece, אני מחזיק ספירה: כמה peers מחזיקים אותו.

כשאני צריך לבחור piece חדש להוריד:

1. מתחשב רק ב-pieces שעוד לא הורדתי.
2. מוצא מהו ערך המינימום של "כמות peers שיש להם את ה-piece".
3. בונה "סט rarest" – כל החלקים שהספירה שלהם שווה למינימום הזה.
4. מתוכנן אני בוחר חלק אחד באקראי, כדי שלא כל ה-clients בעולם יבקשו את אותו piece בדיוק באותו זמן.

סיבוכיות

- עדכון ספירה עם כל הודעת have הוא $O(1)$.
- חישוב הסט הנדיר ביותר הוא $O(n)$ עבור pieces n , וזה סביר לחלוטין לגודל הפרויקטים שאני עובד איתם.

טבלת SWOT – rarest-first

נקודות חולשה (Weaknesses)	נקודות חוזק (Strengths)
S דורש לנהל מבנה נתונים שמחזיק ספירה לכל piece, ולעבור עליו כדי למצוא rarest $\Rightarrow$ יותר חישוב מבחירה אקראית.	S שומר על זמינות החלקים הנדירים לאורך זמן, ומפחית את הסיכון להיתקע על "piece אחרון" שקשה להשיג.
W רגיש לכך שהמידע על החלקים מעודכן; אם חלק מה-peers לא שולחים have/bitfield בזמן, התמונה עלולה להיות לא מדויקת.	W האלגוריתם יכול להיות פחות יעיל בקבצים עצומים אם מממשים אותו בצורה נאיבית (ללא אופטימיזציות).
O ניתן לשפר את היעילות בעזרת bucket, cache-ים או מבני נתונים נוספים שמקטינים את הצורך בסריקות מלאות.	O אפשר לשלב אותו עם מדיניות נוספת, למשל להתחיל בהורדת מספר MB ראשונים באופן סדרתי לצורך סטרימינג ואז לעבור ל-rarest-first.

T peers זדוניים יכולים לשקר לגבי אילו pieces יש להם, ולשבש את ההחלטות (למשל לדווח שיש להם חלקים נדירים כשאינן להם). קיימים אלגוריתמים מורכבים יותר (כמו network coding) שיכולים לעקוף את rarest-first בהיבטים מסוימים, אבל הם הרבה יותר מסובכים ליישום.

7.2 אלגוריתם (Tit-for-Tat) (choke/unchoke) – בחירת peers

איך אני מיישם אותו

1. אני מחזיק רשימה של כל ה-peers שאני מחובר אליהם, ולכל אחד אני עוקב אחרי כמות הנתונים שהוא העלה לי במהלך פרק זמן מוגדר (interval). כל כמה שניות (למשל כל 10 שניות):
 1. מסנן רק peers שמצהירים שהם "מעוניינים" (interested) לקבל ממני נתונים.
 2. ממין אותם לפי כמה הם העלו לי בתקופה האחרונה (upload אליי).
 3. בוחר את K ה-peers עם התרומה הגדולה ביותר ומשאיר אותם במצב unchoked (הם מקבלים ממני נתונים).
 4. בנוסף, אני בוחר peer אחד נוסף באופן אקראי (optimistic unchoke) גם אם הוא עדיין לא תרם, כדי לתת הזדמנות לגלות שותפים חדשים טובים.

בצורה הזו, מי שתורם לי יותר, נהנה מהורדה מהירה יותר ממני. מי שלא נותן כמעט כלום – לרוב יישאר "חנוק". זוהי גרסה מעשית של Tit-for-Tat

סיבוכיות

החלק הכי "כבד" הוא המיון של רשימת ה-peers כל INTERVAL, וזה $O(m \log m)$ עבור peers m. במספרים שאני מצפה להם (עשרות peers לכל היותר), זה זניח.

טבלת SWOT – choke/unchoke

נקודות חולשה (Weaknesses)	נקודות חוזק (Strengths)
peers עם רוחב פס חלש עלולים להיפגע – ייתכן שיהיה להם קשה להוכיח "תרומה" ולקבל unchoke.	S מעודד שיתוף: peers שמעלים יותר, לרוב יורידו יותר מהר – יש להם אינטרס להיות "חברים טובים".
קיימים לקוחות "חכמים" מדי" (BitTyrant) שמנסים לנצל את הכללים לטובתם.	W האלגוריתם יחסית פשוט ליישום ונבחן רבות במחקרים מעשיים על BitTorrent.
peers זדוניים יכולים "לזייף" תרומה, למשל לשלוח נתונים לא מועילים אם לא בודקים hash.	O אפשר לכוון את מספר ה-unchoked ואת פרק הזמן בין העדכונים כדי להתאים לרשתות שונות (בית, מוסד, ענן).
במצבי קיצון של חוסר איזון ברוחב פס, יכול להיווצר מצב של חוסר הוגנות בין משתמשים שונים.	T אם רבים ברשת עוברים לאלגוריתמים חמדניים (כמו BitTyrant), התמריצים המקוריים של Tit-for-Tat יכולים להיחלש.

7.3 השוואה למדיניות בסיסית – Random Piece / Round-Robin

כדי לכמת את "ההישג" של האלגוריתמים שלי, אני רוצה להשוות אותם למדיניות בסיסית יותר:

- **Random Piece Selection** – בחירת piece אקראי מתוך כולם.
- **Round-Robin Peer Selection** – חלוקת הבקשות בין peers בתור, בלי קשר לתרומה שלהם.

מדובר באלגוריתמים מאוד פשוטים – קל לממש אותם ולנתח אותם – אבל מחקרים מראים שהם פחות טובים מבחינת זמינות חלקים ותמריצים לשיתוף לעומת rarest-first ו-choke.

SWOT קצר – Random / Round-Robin

קטגוריה	ניתוח
S	קל למימוש, קל להסביר, מתאים כקו בסיס להשוואה.
W	לא מתחשב בנדירות חלקים ולא יוצר תמריצים אמיתיים לשיתוף; עלול להשאיר חלקים נדירים "תקועים".
O	אפשר להשתמש בהם כבסיס לניסויים ולהראות בצורה ברורה למה rarest-first ו-choke משפרים ביצועים.
T	ב-SWARM אמיתי עלולים להוביל לביצועים גרועים ולמצבים שבהם לא מצליחים להשלים את הקובץ כולו.

8. אבטחת מידע

8.1 Hash per Piece – שלמות מידע

קובץ ה-**torrent** כולל רשימת SHA-1 hash לכל piece. בכל פעם שסיימתי להוריד piece שלם (אוסף הבלוקים שמרכיבים אותו), אני מחשב מחדש את ה-hash על הנתונים שהורדתי ומשווה ל-hash הצפוי. אם הם לא תואמים, אני דוחה את החלק, לא כותב אותו לקובץ, ומסמן את ה-peer כבעייתי.

8.2 התמודדות עם peers זדוניים או שגויים

אני בודק שההודעות שאני מקבל עומדות במבנה של ה-spec (למשל שה-length תקין, שסוג ההודעה מותר בשלב הזה ב-state machine וכו'). אם peer שולח הודעות לא חוקיות או חריגות (למשל הודעה ארוכה מדי או **piece** שלא התבקש), אני מנתק אותו ושומר לוג. זה מגן מפני קריסות לא צפויות ומפני ניצול פרוטוקול.

בנוסף, אני מתכנן להשתמש ב-timeouts ולהגביל גודל הודעות מקסימלי כדי להקטין סיכון ל-flooding.

8.3 פרטיות מול ה-Tracker

בזמן announce, אני שולח ל-tracker מידע על ההורדה שלי: **peer_id**, **info_hash**, ה-IP שלי, ויש גם סטטיסטיקות **uploaded** ו-**downloaded**. אני מנתח איזה מידע זה חושף עליי (למשל איזה torrents אני מוריד) ואיזה סיכונים זה מייצר. בעתיד אפשר להרחיב את זה ללימוד מנגנונים כמו DHT או Peer Exchange שמפחיתים תלות ב-tracker יחיד.

9. פיתוחים עתידיים

אחרי שאסיים את גרסת הבסיס, אני רואה כמה כיוונים אפשריים לפיתוח:

- להוסיף תמיכה ב-**UDP trackers** וב-BEP-ים נוספים שקשורים אליהם.
- לשלב **DHT (Distributed Hash Table)** למציאת peers גם בלי Tracker מרכזי.
- להרחיב את המנוע כך שיתמוך גם ב-**multi-file torrents** ובגרסת v2 BitTorrent.
- להוסיף **הצפנת תעבורה** בין peers (Protocol Encryption) לצמצום יכולת ניתור התעבורה.
- לשפר את ה-GUI ב-Java עם גרפים בזמן אמת, פילטרים, והצגת לוגים.

10. מוד נתונים (JSON + SQLite) – Persistency

בשלב הראשון אני לא מתכנן להשתמש במסד נתונים "כבד" כמו MySQL/PostgreSQL, כי המערכת שלי היא תוכנת Desktop שפועלת מקומית והנתונים שהיא מייצרת הם בעיקר סטטוס הורדות, היסטוריה וסטטיסטיקות. עם זאת, חשוב לי שהמערכת תהיה עמידה: שגם אם התוכנה נסגרת באמצע, ניתן לשחזר את ההורדות ואת המידע שנאסף.

לכן אני מחלק את שכבת השמירה לשני סוגים של נתונים, שכל אחד נשמר בצורה שמתאימה לו:

שמירת מצב הורדה בזמן אמת – JSON

מצב ההורדה (state) של כל torrent יישמר בקובצי JSON מקומיים. כאן מדובר בנתונים שמתעדכנים הרבה: אילו pieces כבר הושלמו, אילו חסרים, מה ההתקדמות, אילו peers מחוברים, ומה המצב של ההורדה (Running / Paused / Completed / Cancelled). שמירה כ-JSON מאפשרת כתיבה מהירה, קריאה פשוטה, ונוחות לדיבוג ולתיעוד.

דוגמאות לנתונים שישמרו ב-JSON:

- torrentId, info_hash, שם הקובץ, גודל כולל, piece_length
- סטטוס לכל piece (הושלם/חסר)
- נתוני התקדמות: downloaded, uploaded, left
- רשימת peers אחרונים שנצפו (IP, port) וסטטוס חיבור בסיסי
- זמני התחלה/עצירה ושמירה אחרונה (last_saved)

המטרה של ה-JSON היא לא "להחליף DB", אלא לאפשר Recovery ושחזור מצב בצורה אמינה ופשוטה.

היסטוריה וסטטיסטיקות – SQLite

בנוסף ל-JSON, אשתמש ב-SQLite כמסד נתונים מקומי קל משקל לצורך ניהול מידע היסטורי וסטטיסטיקות שלא חייב להתעדכן כל שנייה, אבל כן חשוב לשמור אותו לאורך זמן. SQLite מתאים כאן כי הוא לא דורש שרת, עובד כקובץ אחד, ומאפשר שאילתות נוחות לצורך דו"חות והצגת היסטוריה ב-GUI.

דוגמאות לנתונים שישמרו ב-SQLite:

- היסטוריית הורדות: מתי התחילה/הסתיימה הורדה, גודל, זמן כולל, סטטוס סופי
- סטטיסטיקות ביצועים: מהירות ממוצעת, מהירות שיא, מספר peers ממוצע
- נתוני אלגוריתמים לצורך ניתוח: כמה פעמים נבחר כל piece כ-rarest, כמה choke/unchoke בוצעו לאורך זמן
- Log בסיסי של אירועים חשובים (למשל peers שניתקו/שגיאות hash)

באופן הזה המערכת משלבת בין JSON לשחזור מצב מהיר לבין SQLite לתיעוד, ניתוח והצגת נתונים בצורה מסודרת, בלי להעמיס על הארכיטקטורה ובלי להכניס DB כבד שלא באמת נחוץ לפרויקט.

11. לוח זמנים

שלב	תקופה	משך משוער	תיאור הפעילויות המרכזיות	תוצרים
1	ינואר 2026 – אמצע פברואר 2026	כ-6 שבועות	לימוד והעמקה בפרוטוקול BitTorrent, פורמט Bencode, Tracker ו-Peer Wire Protocol. חקר אלגוריתמים rarest-first ו-choke/unchoke. סקירת פתרונות קיימים וגיבוש ארכיטקטורה כללית.	רקע תיאורטי מלא, הגדרת הבעיה האלגוריתמית, החלטות טכנולוגיות
2	אמצע פברואר 2026 – תחילת מרץ 2026	כ-2-3 שבועות	תכנון המערכת: חלוקה למודולים, תכנון REST API, הכנת Use Case Diagram ותרחישי UML, תכנון JSON Persistency (+ SQLite) ומבני נתונים.	תרשימי ארכיטקטורה, Use Case Diagram, תכנון API
3	מרץ 2026 – תחילת אפריל 2026	כ-4-5 שבועות	מימוש מנוע BitTorrent בפייטון: Bencode, קריאת קובצי torrent, Tracker Client, Peer Connections, מימוש rarest-first ו-choke/unchoke, בדיקות hash.	מנוע BitTorrent עובד (CLI), הורדות תקינות

4	תחילת אפריל 2026 – אמצע אפריל 2026	כ-2 שבועות	פיתוח REST API, שמירת מצב הורדה ב-JSON, מימוש SQLite (torrents, sessions, events, stats), בדיקות שחזור מצב.	API יציב, Persistency מלאה
5	אמצע אפריל 2026 – סוף אפריל 2026	כ-2 שבועות	פיתוח GUI ב-Java (JavaFX/Swing), חיבור ל-API, הצגת סטטוס והיסטוריה, מימוש Pause / Resume / Cancel.	אפליקציית Desktop עובדת מקצה לקצה
6	סוף אפריל 2026 – תחילת מאי 2026	כ-1-2 שבועות	בדיקות מערכת, ניסויי ביצועים, השוואת אלגוריתמים, איסוף נתונים וניתוח תוצאות.	תוצאות ניסוי, טבלאות השוואה ומסקנות
7	תחילת מאי 2026 – 14.05.2026	כ-2 שבועות	כתיבת ספר הפרויקט לפי תבנית משרד החינוך, שילוב תרשימים וקוד, כתיבת מדריך משתמש והכנה להצגה בעל-פה.	ספר פרויקט מלא, קוד מוכן להגשה

12. רשימת מאמרים (אלגוריתמיקה ו-BitTorrent)

1. **Bram Cohen**, *Incentives Build Robustness in BitTorrent* (2003) – מקור הרעיונות של rarest-first ו-Tit-for-Tat בפרוטוקול BitTorrent.
2. **A. Legout, G. Urvoy-Keller, P. Michiardi**, *Rarest First and Choke Algorithms Are Enough* (IMC 2006) – מאמר שמציג תוצאות ניסוייות ומראה ששילוב rarest-first ו-choke נותן ביצועים טובים והוגנות ברמת ה-SWARM.
3. **A. Legout et al.**, *Understanding BitTorrent: An Experimental Perspective* – ניתוח מעשי של התנהגות BitTorrent ברשת אמיתית, כולל תיאור מפורט של האלגוריתמים והפצת חלקים.
4. **M. Piatek et al.**, *Do Incentives Build Robustness in BitTorrent? (BitTyrant)*, NSDI 2007 – מציג לקוח אגרסיבי בשם BitTyrant ומנתח איך אפשר לנצל את אלגוריתם ה-choke הקלאסי, ומה המשמעות מבחינת תמריצים.
5. **F. Azzedin**, *Modeling BitTorrent choking algorithm using game theory*, Future Generation Computer Systems, 2016 – מודל מתמטי של אלגוריתם ה-choke באמצעות תורת המשחקים והשלכות על יעילות והוגנות.
6. **BEP-3 – The BitTorrent Protocol Specification** – מסמך ה-spec הרשמי שמגדיר את peer wire protocol, handshake והודעות.

7. מסמכי **Tracker Protocol / UDP Tracker / DHT** – מסבירים את פרטי ה-announce, פורמטי התשובות והרחבות עתידיות (UDP tracker, DHT).
8. "But how does torrenting actually work?", dev.to – מדריך מעשי שמסביר את הזרימה בין tracker, peers ו-pieces מנקודת מבט של מפתח.

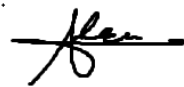
13. מילון מושגים

מושג	הסבר קצר וברור
BitTorrent	פרוטוקול לשיתוף קבצים שבו משתמשים מחליפים חלקים זה עם זה (P2P) במקום שרת מרכזי.
Torrent / .torrent	קובץ מטא-דאטה שמכיל מידע על הקבצים להורדה: pieces, hashes, כתובת tracker וכו'.
Bencode	פורמט קידוד פשוט (int, string, list, dict) שמשמש בקובצי .torrent ובטקרים.
(Info Dictionary (info	החלק המרכזי ב-torrent המתאר את מבנה הקבצים; עליו מחשבים .info_hash
info_hash	SHA-1 על מילון ה-info; מזהה ייחודי לטורנט בכל העולם.
Tracker	שרת שמחזיר רשימת peers פעילים לטורנט מסוים בתגובה לבקשת .announce

Announce Request	בקשת HTTP שהלקוח שולח ל-tracker עם peer_id, info_hash וסטטיסטיקות הורדה.
Peer	כל מחשב שמשתתף בהורדה/העלאה של חלקי הקובץ.
Seeder	peer שיש לו 100% מהקובץ ומעלה לאחרים בלבד.
Leecher	peer שעדיין מוריד אבל גם משתף חלקים תוך כדי.
Swarm (סוורם)	כל ה-seeders + leechers (peers) שמורידים/משתפים את אותו טורנט כרגע.
Peer Wire Protocol	הפרוטוקול מעל TCP שמגדיר את התקשורת בין peers (הודעות).
Handshake	שלב ראשוני שבו שני peers מחליפים info_hash ו-peer_id כדי לוודא שהם באותו טורנט.
Bitfield	הודעה שמספרת אילו pieces יש ל-peer בצורת ביטים.
Piece	חלק מהקובץ (למשל 256KB); הקובץ מחולק לעשרות/מאות כאלה.
Block	חלק קטן יותר מתוך piece (למשל 16KB) שנשלח בתקשורת.
Hash per Piece	בדיקת תקינות: השוואת SHA-1 של piece ל-hash שבקובץ ה-torrent.
Piece Selection	החלטה איזה piece כדאי להוריד עכשיו.
Rarest-First	אלגוריתם שבוחר קודם את החלקים שהכי מעט peers מחזיקים.
Peer Selection	החלטה מאילו peers להוריד ומי יקבל רוחב פס ממני.
Choke / Unchoke	מנגנון המאפשר/חוסם הורדה מ-peer בהתאם לתרומה שלו (תן וקח).
Tit-for-Tat	עיקרון: מי שמעלה לי יותר — יקבל ממני יותר; מונע free-riding.
Optimistic Unchoke	פתיחה זמנית של ערוץ ל-peer חדש כדי לבדוק אם הוא מועיל.
PeerConnection	מודול שמנהל חיבור TCP אחד ל-peer (handshake), הודעות, קבלת בלוקים.
Piece Manager	מודול שמנהל אילו חלקים יש/אין לי וכמה peers מחזיקים כל חלק.

המוח של ההורדה: בחירת peers, בחירת pieces וניהול הזרימה.	Download Manager
מערכת מצבים שמגדירה מה מותר/אסור לשלוח בכל שלב בפרוטוקול.	State Machine
תקיפה/התנהגות שבה peer מציף בהודעות כדי להעמיס או להפיל חיבור.	Flooding
פרק זמן מרבי שבו מותר ל-peer לא להגיב לפני ניתוק.	Timeout
ממשק (localhost HTTP) שדרכו ה-Java GUI מתקשר עם מנוע הפייטון.	REST API
הממשק הגרפי להצגת הורדות, מהירות, אחוזים וכפתורי שליטה.	(GUI (JavaFX/Swing
ערוץ תקשורת קבוע ודו-כיווני דרכו מועבר ה-peer wire protocol.	TCP Socket
דרך "דחוסה" שבה tracker שולח רשימת peers כמבנה בינארי.	Compact Peers Format
היחס בין משתפים מלאים לבין מורידים — משפיע על מהירות הסוורר.	Seeder/Leecher Ratio
מודל רשת שבו מחשבים משתפים ישירות זה עם זה.	P2P
שיטה למציאת peers ללא Tracker — מפזרת מידע בטבלה מבוזרת.	DHT
גרסה של טרקר שעובדת ב-UDP (פשוטה ומהירה יותר מ-HTTP).	UDP Tracker
מידע תיאורי שמגיע מתוך קובץ ה-hashes, lengths torrent וכו').	Metadata
כמות הנתונים שאפשר לשלוח/לקבל בשנייה (מהירות חיבור).	Bandwidth
האם האלגוריתם מחלק משאבים בצורה הוגנת בין המשתתפים.	Fairness
peer שמנסה רק להוריד ולא להעלות.	Free-rider
מסמך שמגדיר חלק מהתקנים/שיפורים של BitTorrent (למשל 3-BEP).	BEP – BitTorrent Enhancement Proposal

חתימת הסטודנט :



חתימת המנחה :



חתימת רכז המגמה :

